CSA47 - Deep Learning

Assignment 3: CO3 - Recurrent and Recursive Neural Networks: Evaluation, Tuning and Debugging

Field	Details
Student name	A HARITHA
Register number	192321147
Mode	Individual submission, 1 week, 50 marks
Department	Information Technology, SIMATS Engineering

This is an individual submission. The assignment describes two roles (Role A and Role B); both roles are completed by the same student. The initial design proposal in Q7 is marked for insertion of the original draft, kept unedited. Experimental values depend on the random seed and hardware; where a run has not been recorded, the table cell is left as a blank to be filled from the experiment log, and no result is claimed.

Q1 - Understanding Sequence Representation and Recurrent Processing (6 marks)

Part A - Sequence representation and recurrent processing

Dataset. Sensor-based fault detection. Each example is a sequence X = [x_1, ..., x_T], where x_t is a vector of d sensor readings at time step t. The target y is the class of the whole sequence (normal or abnormal). T is the sequence length and d is the feature dimension (d = 3 in the experiment below).

Role A - Sequence design and mathematical analysis

- **Roles of the symbols.** x_t is the input at step t. h_t is the hidden state: a learned summary of x_1 ... x_t. y_t is the output; for a whole-sequence label only the final output is used.
- **Equations.** The same weights are reused at every step:

```
h_t = tanh(W_xh x_t + W_hh h_(t-1) + b_h) y_t = g(W_hy h_t + b_y) (g = sigmoid for binary output)
```

Unfolded structure. The cell is copied once per time step, and each copy passes its hidden state to the next:

```
  x1          x2          x3                  xT
   |           |           |                   |
   v           v           v                   v
[cell]--h1-->[cell]--h2-->[cell]--h3--> ... --[cell]--hT--> y
  h0=0                                                  (final state -> classifier)
```

- **Hidden state.** Because h_t depends on h_(t-1), information from early inputs reaches later steps through the chain of states, so h_T summarizes the whole sequence.
- **Sequence length.** A longer T gives more context but increases computation linearly, and the gradient must travel through more steps. Longer is only better if the added steps carry useful information.
- **Long-term dependency example.** A fault signature in the first 5 readings determines the label of a 200-step sequence. The gradient from the output must pass through about 200 tanh and weight multiplications before reaching the early inputs, so it tends to vanish and the network learns mainly from recent steps.

Worked state update (illustrative numbers). Scalar input, W_xh = 0.6, W_hh = 0.9, b_h = 0, h_0 = 0, and x = [0.8, -0.4, 0.5]:

```
h1 = tanh(0.6 x 0.8 + 0.9 x 0) = tanh(0.480) = 0.446 h2 = tanh(0.6 x (-0.4) + 0.9 x 0.446) = tanh(0.162) = 0.160 h3 = tanh(0.6 x 0.5 + 0.9 x 0.160) = tanh(0.444) = 0.417
```

The first input affects h_3 only through the chain h_1 -> h_2 -> h_3, each step multiplied by W_hh and a tanh derivative (at most 1). With derivatives below 1 this repeated multiplication weakens distant influence.

Parameter counts (hidden size 64, d = 3). Simple RNN: 64x3 + 64x64 + 64 = 4,352. LSTM: 4 x 4,352 = 17,408. GRU (Keras default, reset_after = True): 13,248. The parameter count does not depend on T, but compute time does.

Role B - Experimental investigation

Design. Train the same Simple RNN with T = 10, 50 and 100, using the same data split, hidden size 32, Adam (lr 0.001), batch size 32, 15 epochs and 3 random seeds. Record the results in this table:

Sequence length T	Train loss	Val loss	Val accuracy	Training time (s)
10	_____	_____	_____	_____
50	_____	_____	_____	_____
100	_____	_____	_____	_____

Interpretation guide. If validation loss rises with T while training loss falls, longer sequences are causing overfitting or harder optimization. If time grows roughly linearly with T, the cost-benefit trade-off is clear. If accuracy stays flat, the extra steps add no useful context.

```
import numpy as np, time
from tensorflow import keras
from tensorflow.keras import layers

def make_data(n, T, d=3, seed=0):
    rng = np.random.default_rng(seed)
    X = rng.normal(0, 1, (n, T, d)).astype('float32')
    y = rng.integers(0, 2, n).astype('float32')
    X[y == 1] += np.linspace(0, 1.5, T)[None, :, None]   # abnormal class drifts upward
    return X, y

for T in (10, 50, 100):
    X, y = make_data(2000, T)
    m = keras.Sequential([layers.Input(shape=(T, 3)), layers.SimpleRNN(32),
                          layers.Dense(1, activation='sigmoid')])
    m.compile('adam', 'binary_crossentropy', metrics=['accuracy'])
    t0 = time.time()
    h = m.fit(X, y, validation_split=0.2, epochs=15, batch_size=32, verbose=0)
    print(T, round(h.history['loss'][-1], 3), round(h.history['val_loss'][-1], 3),
          round(h.history['val_accuracy'][-1], 3), round(time.time() - t0, 1))
```

Part B - Recurrent versus recursive processing

- **Role A - recurrent network.** An RNN processes inputs one time step at a time, in order. Its state at step t is a function of the previous state and the current input. Time steps are the unit of computation: a sequence of length T takes T sequential updates. Parameters are shared across time, so the model learns one transition rule that applies at every position. Suitable applications: sensor series, speech, forecasting and text where word order matters.
- **Role B - recursive network.** A recursive network composes child representations into a parent representation following a tree, so representations are built bottom-up from phrases to sentences. The same composition weights are used at every node. A parse tree of a sentence of n words has depth about log2(n) for balanced trees, so composition is hierarchical rather than step-by-step. Suitable applications: sentiment on parsed sentences, and nested or syntactic structures.

Evaluation. For a sentence input, a sequential RNN is more appropriate when word order is the main signal and no reliable parse is available: for example, short noisy chat messages, ungrammatical text, or a task where the position of words matters but phrase boundaries are unknown. A recursive representation is more appropriate when the sentence has a trustworthy syntactic structure and meaning comes from how phrases combine, for example negation scope in 'not very good', where the modifier must apply to a phrase rather than to the next word. The decision follows the data's structure: order-driven data suits an RNN; hierarchy-driven data with reliable trees suits a recursive model.

Q2 - Evaluating RNN Performance Using Multiple Metrics (7 marks)

Role A - Metric investigation

Using a held-out test set, the confusion matrix counts TP, TN, FP and FN (abnormal = positive class):

```
Accuracy = (TP + TN) / (TP + TN + FP + FN) Precision = TP / (TP + FP) (how many
predicted abnormal are correct) Recall = TP / (TP + FN) (how many actual abnormal are
detected) F1 = 2 x Precision x Recall / (Precision + Recall)
```

Illustrative counts (not experimental results): TP = 36, TN = 48, FP = 6, FN = 10 out of 100 sequences.

	Predicted abnormal	Predicted normal
Actual abnormal	TP = 36	FN = 10
Actual normal	FP = 6	TN = 48

Metric	Calculation	Value
Accuracy	(36 + 48) / 100	84.0%
Precision	36 / 42	85.7%
Recall	36 / 46	78.3%
F1-score	2 x 0.857 x 0.783 / (0.857 + 0.783)	81.8%

Training versus validation. High error on both sets indicates underfitting. Strong training metrics with weaker validation metrics indicate overfitting. Use the gap, the learning curves and the metric at each epoch to decide. Accuracy alone does not reveal which of these is happening on a rare class.

Why one metric is not enough. Accuracy weights all errors equally and can be dominated by the majority class. Precision ignores missed cases; recall ignores false alarms; F1 balances the two but hides the cost of each error. Report at least the confusion matrix, precision, recall and F1 for the abnormal class, plus PR-AUC when classes are imbalanced.

```
import numpy as np
from sklearn.metrics import (confusion_matrix, accuracy_score, precision_score,
                             recall_score, f1_score, average_precision_score)

def evaluate(y_true, y_prob, threshold=0.5):
    y_pred = (y_prob >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    print('TP=%d TN=%d FP=%d FN=%d' % (tp, tn, fp, fn))
    print('accuracy %.3f precision %.3f recall %.3f f1 %.3f pr_auc %.3f' % (
        accuracy_score(y_true, y_pred), precision_score(y_true, y_pred, zero_division=0),
        recall_score(y_true, y_pred, zero_division=0), f1_score(y_true, y_pred, zero_division=0),
        average_precision_score(y_true, y_prob)))

# check against the illustrative counts above
y_true = np.array([1]*36 + [1]*10 + [0]*6 + [0]*48)
y_pred = np.array([1]*36 + [0]*10 + [1]*6 + [0]*48)
print(round(f1_score(y_true, y_pred), 4), accuracy_score(y_true, y_pred))   # 0.8179  0.84
```

Role B - Sequence-level performance analysis

- **Metric choice.** For a numeric forecast, use MAE (average absolute error in original units) and RMSE (penalizes large errors more). For an event classifier with rare events, use F1 or PR-AUC at a threshold chosen on validation data.
- **Performance by sequence length.** Group test sequences into length bins (for example 20-40, 40-60, 60-80) and compute the same metric per bin. Record validation loss and task metric together. A bin where performance drops shows where the model stops using long context.
- **Loss falls, task metric does not.** Cross-entropy can improve when predicted probabilities become better calibrated, even if no predicted class changes. It can also improve only on already easy examples. Check the thresholded metric alongside the loss.

- **Link to the application.** If a missed fault is costly, recall matters most and the threshold should be lowered; if false alarms stop production, precision matters most. The metric should reflect that cost, not the loss alone.

Evaluation. A model with 95% accuracy that performs poorly on the minority class should not be accepted automatically. Before accepting it, we would require: (1) the confusion matrix and minority-class precision, recall and F1; (2) PR-AUC and a comparison with a trivial majority-class baseline, which already reaches 95% when the minority class is 5%; (3) validation and test results on a separate, untouched test set, and ideally on another time period or site; (4) the threshold chosen on validation data, with its precision-recall trade-off; (5) calibration of probabilities; (6) the real cost of false negatives and false positives agreed with the domain experts; and (7) results across several seeds, with spread reported, so the result is not luck. Be ready to defend this answer in a viva.

Q3 - Selecting Hyperparameters for an RNN (7 marks)

Role A - Hyperparameter design

Hyperparameter	Candidate values	Why it matters
Learning rate	0.001, 0.0005, 0.0001	Too high: unstable or diverging; too low: slow training
Hidden units	32, 64, 128	Capacity: too few underfit, too many overfit and cost more
Recurrent layers	1, 2	Depth gives more abstraction but harder optimization
Sequence length	Fixed T, or cap at 99th percentile	More context versus more noise, compute and gradient distance
Batch size	32, 64	Gradient noise versus throughput and memory
Dropout	0.0, 0.2, 0.4	Regularization; too much causes underfitting

Controlled experiment. Keep the split, preprocessing and metrics fixed. Start from a baseline (lr 0.001, 64 units, 1 layer, batch 32, dropout 0.2), then vary one factor at a time, or run a structured random search. Each configuration is repeated with a fixed seed set so that run-to-run variation is visible.

Likely underfitting: very low learning rate with few epochs, 32 units with two layers and heavy dropout.
Likely overfitting: 128 units, two layers, dropout 0.0, a long sequence on a small dataset.

Role B - Hyperparameter evaluation

Grid size: 3 learning rates x 3 unit sizes x 2 layer counts x 2 batch sizes x 3 dropout values = 108 configurations. A random search of 20 configurations is usually enough, because learning rate and capacity dominate performance.

Trial	LR	Units	Layers	Batch	Dropout	Val loss	Val F1	Time (s)
1	0.001	64	1	32	0.2	_____	_____	_____
2	0.0005	64	1	32	0.2	_____	_____	_____
3	0.001	128	2	64	0.4	_____	_____	_____
...	...	...	...	...	...	...	...	...

Analysis to record. Compare convergence speed (epochs to reach a validation plateau), stability (spread of the validation curve), and whether the best training score is also the best validation score. Experimental analysis of at least two hyperparameters should show a plot or table of one factor against validation loss, with all others held fixed. Select the configuration with the best validation performance at acceptable time, then evaluate it once on the test set.

```
import itertools, random, time
from tensorflow import keras
from tensorflow.keras import layers

def build(T, d, lr, units, n_layers, dropout):
    m = keras.Sequential([layers.Input(shape=(T, d))])
    for i in range(n_layers):
        m.add(layers.LSTM(units, return_sequences=(i < n_layers - 1)))
        m.add(layers.Dropout(dropout))
    m.add(layers.Dense(1, activation='sigmoid'))
    m.compile(keras.optimizers.Adam(lr), 'binary_crossentropy', metrics=['accuracy'])
    return m

grid = list(itertools.product([1e-3, 5e-4, 1e-4], [32, 64, 128], [1, 2], [32, 64], [0.0, 0.2, 0.4]))
random.seed(7)
results = []
for lr, units, n_layers, batch, drop in random.sample(grid, 20):
    m = build(X_train.shape[1], X_train.shape[2], lr, units, n_layers, drop)
    stop = keras.callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
    t0 = time.time()
    h = m.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=40,
              batch_size=batch, callbacks=[stop], verbose=0)
    results.append((min(h.history['val_loss']), lr, units, n_layers, batch, drop, time.time() - t0))
```

```
results.sort()
for r in results[:5]:
    print('val_loss=%.4f lr=%g units=%d layers=%d batch=%d drop=%.1f time=%.0fs' % r)
# Retrain the best configuration and evaluate ONCE on (X_test, y_test).
```

Joint design question. If training performance is excellent but performance on unseen sequences is poor, first check for data leakage and split quality (the same sensor or machine in both training and test sets, duplicated sequences). If the split is clean, investigate regularization and capacity next: dropout and L2 weight decay, fewer units or layers, early stopping, and more representative training data. The learning rate is tuned if curves are unstable. The reason for this order is that a leaky split can make any hyperparameter look good, so it must be ruled out before tuning. The best training score is not the best generalization score, and the choice must be justified with validation curves, not the training loss.

Q4 - Debugging an RNN: Diagnosing Training Failure (7 marks)

Problematic configuration - the problematic configuration

The problem configuration used a SimpleRNN with a learning rate of 0.5 on unnormalized inputs (scale multiplied by 100). The loss oscillated and then became NaN after several epochs. The steps below follow from that observation.

Role B - Diagnosis and correctness

Symptom	Likely cause	Check	Corrective action
Loss becomes NaN/Inf	Exploding gradients, LR too high, log(0), bad input values	First NaN tensor, gradient norm, input min/max	Lower LR, clip gradients, clip probabilities, clean data
Loss does not decrease	LR too low, unscaled inputs, wrong labels or loss	Overfit a tiny batch, check labels	Scale inputs, adjust LR, fix pipeline
Good early steps, poor long sequences	Vanishing gradients	Accuracy versus T	LSTM/GRU, shorter sequences, better initialization
Train good, validation poor	Overfitting or leakage	Learning curves, split method	Dropout, L2, early stopping, more data
Both poor	Underfitting or a bug	Shapes, features, capacity	Larger model, better features, fix bug

- **Gradient flow.** In recurrent training the gradient for early steps is a product of T Jacobians. If their norms are above 1 the product explodes; below 1, it vanishes. Both make the update for long-range effects unreliable.
- **Gradient clipping (global norm).** If ||g|| exceeds a threshold tau, the gradient is rescaled: g <- g x tau / ||g||. Example: g = [30, 40], ||g|| = 50. With tau = 5, the clipped gradient is [30, 40] x 5/50 = [3, 4]. The direction is kept and only the size is limited, so one extreme batch cannot destabilize training. Clipping does not fix vanishing gradients; gated cells and shorter sequences do.
- **Implementation error versus design problem.** An implementation error is wrong code, shapes, labels or a missing normalization; it usually shows up as failure to overfit a tiny batch. A design problem is a correct implementation with unsuitable architecture, capacity or training settings, which can be fixed by changing the design.
- **Final debugging decision.** First verify the implementation (data checks, tiny-batch overfit). Then, one factor at a time, lower the learning rate and add clipping. Change the architecture only if the corrected pipeline still fails.

Role A - Experimental and engineering analysis

Run	Change (one factor)	Loss behaviour	Val F1	Time (s)	Stable?
Before	LR 0.5, unnormalized inputs	Oscillates, NaN at epoch ___	_____	_____	No
Fix 1	Standardize inputs (train mean/std)	_____	_____	_____	_____
Fix 2	LR 0.001 + clipnorm 1.0	_____	_____	_____	_____

Analysis to record. Compare the curves before and after each intervention, and decide which change was most effective from the evidence. Clipping adds a small per-step cost; a smaller learning rate increases the number of epochs needed. These are the computational trade-offs to report.

```
import numpy as np, tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

assert not np.isnan(X_train).any() and not np.isinf(X_train).any(), 'NaN/Inf in inputs'
print('labels', np.unique(y_train), 'mean/std', X_train.mean().round(3), X_train.std().round(3))

# Stability fix: clip the global gradient norm and stop at the first NaN
```

```
opt = keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1.0)
model = keras.Sequential([layers.Input(shape=X_train.shape[1:]),
                          layers.SimpleRNN(32), layers.Dense(1, activation='sigmoid')])
model.compile(opt, 'binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=60, batch_size=32,
          callbacks=[keras.callbacks.TerminateOnNaN(),
                     keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True)])

# Gradient norm for one batch
with tf.GradientTape() as tape:
    loss = keras.losses.binary_crossentropy(y_train[:32, None], model(X_train[:32]))
grads = tape.gradient(loss, model.trainable_variables)
print('global gradient norm:', float(tf.linalg.global_norm(grads)))
```

Joint debugging decision - NaN loss after several epochs. (1) Stop training and keep the last finite checkpoint. (2) Locate the first tensor that becomes NaN or Inf: check the inputs, the loss, the gradients and the weights in that order. (3) Check numerical operations such as log(0), division by a near-zero standard deviation, and unclipped probabilities. (4) Inspect the gradient norm per step and the input range. (5) Reduce the learning rate and enable clipping. (6) Rerun the same controlled experiment with the same seed. (7) Only if this fails after the data and training pipeline are verified, consider changing the architecture. The architecture should not be replaced first, because the failure is usually numerical or procedural.

Q5 - RNN versus Recursive Neural Network: Model Selection and Evaluation (6 marks)

Role A - Architecture and representation design

- **RNN input.** A sequence of token or sensor vectors in order. The hidden state h_t is updated at each step and preserves the context seen so far, with order built in.
- **Recursive model.** Leaves are input vectors. Each internal node combines its two children with the same weights: p = tanh(W [c_left ; c_right] + b). The root is the representation of the whole structure.
- **Structural assumptions.** The RNN assumes that order and recency matter, and that a smooth left-to-right dependency is a good model. The recursive model assumes a meaningful tree is available and that phrase-level composition carries the signal.
- **What each preserves.** The RNN preserves sequence position and gradual context; the recursive model preserves hierarchical grouping but may lose the exact order inside a phrase unless the tree encodes it.
- **Computational needs.** RNN cost grows with T and is sequential in time. Recursive cost grows with the number of nodes and needs a parser to build trees, which adds preprocessing cost and error.

```
import numpy as np
d = 4
rng = np.random.default_rng(0)
W = rng.normal(0, 0.5, (d, 2 * d))
b = np.zeros(d)
v = {w: rng.normal(0, 1, d) for w in ['deep', 'learning', 'models']}
def compose(left, right):
    return np.tanh(W @ np.concatenate([left, right]) + b)
p1 = compose(v['deep'], v['learning'])   # phrase 'deep learning'
p2 = compose(p1, v['models'])            # sentence representation
print('sentence vector:', np.round(p2, 3))
```

Role B - Comparative evidence

- **Equivalent evaluation.** Use the same train, validation and test split, the same target definition and the same metrics (accuracy and F1, or MAE/RMSE). The recursive model needs trees for the same examples, so the examples used must have a parse for both models.
- **Record.** Training and validation loss per epoch, validation metrics, training time, inference time per example, and the cost of parsing for the recursive model. Repeat with 3 seeds and report the spread.
- **Strengths and weaknesses.** The RNN needs no parser and is easy to batch, but may struggle with long-range structure. The recursive model captures composition directly, but depends on parse quality and is harder to batch because each tree has a different shape.
- **Decision rule.** Choose the RNN unless the recursive model gives a stable validation gain that outweighs its parsing and training cost.

Cross-understanding. *Role B:* a recursive model is useful when the input has a natural hierarchy, because its composition follows the structure, so a phrase such as a negated adjective is combined before it meets the rest of the sentence. *Role A:* the RNN is preferred when the recursive model gives no measured validation advantage; the evidence needed is equal or better validation metrics across seeds, with acceptable cost and no dependence on a parser that may fail on the target data.

Q6 - Advanced Sequence-Model Mini-Investigations (7 marks)

Role A - Long-term dependency investigation

Task. The label is the sign of the first element of the sequence; all later elements are random noise, so the model must remember the first step until the end.

```
import numpy as np, time
from tensorflow import keras
from tensorflow.keras import layers

def first_step_task(n, T, seed=0):
    rng = np.random.default_rng(seed)
    X = rng.normal(0, 1, (n, T, 1)).astype('float32')
    y = (X[:, 0, 0] > 0).astype('float32')    # depends on time step 1 only
    return X, y

for T in (10, 30, 60):
    X, y = first_step_task(4000, T)
    for name, cell in [('SimpleRNN', layers.SimpleRNN), ('LSTM', layers.LSTM), ('GRU', layers.GRU)]:
        m = keras.Sequential([layers.Input(shape=(T, 1)), cell(32), layers.Dense(1, activation='sigmoid')])
        m.compile(keras.optimizers.Adam(1e-2, clipnorm=1.0), 'binary_crossentropy', metrics=['accuracy'])
        t0 = time.time()
        h = m.fit(X, y, validation_split=0.25, epochs=12, batch_size=64, verbose=0)
        print(T, name, round(h.history['val_accuracy'][-1], 3), round(time.time() - t0))
```

Results table (to be filled from the run log; chance accuracy is 50%):

T	SimpleRNN val acc	LSTM val acc	GRU val acc	SimpleRNN time (s)
10	_____	_____	_____	_____
30	_____	_____	_____	_____
60	_____	_____	_____	_____

Vanishing gradient. During backpropagation through time, the gradient reaching step 1 is a product of recurrent Jacobians. With a weight near 0.9 and a tanh derivative near 0.8, each step multiplies the gradient by about 0.72, so after 30 steps the factor is 0.72^30, about 5 x 10^-5. The early input therefore has almost no effect on the update. A rising T with accuracy falling toward 50% is the observable evidence of this.

Intervention. Use an LSTM or GRU, whose additive cell-state update c_t = f_t * c_(t-1) + i_t * c~_t lets the gradient pass through steps with a gate near 1 rather than being repeatedly squashed. Gradient clipping helps exploding gradients but not vanishing ones. Compare before and after at the same T, and explain the result: if the gated model improves at T = 30 and 60, the gating is responsible; if it does not, the training length or learning rate should be examined.

Role B - Debugging and model refinement investigation

Controlled problem. The learning rate is set to 0.5 (other settings equal to the stable baseline). Symptom: loss oscillates and may become NaN. Hypothesis: the step size is too large for the gradient scale. Test: one change at a time.

Experiment	Change (one factor)	Symptom observed	Evidence to record
Baseline	Normalized inputs, LR = 0.001	Stable decreasing loss	Train/val loss, F1, time
Problem 1	LR raised to 0.5	Oscillation or NaN	Loss curve, gradient norm
Fix 1	LR back to 0.001	Stable training returns	Same metrics as baseline
Problem 2	Inputs not normalized (scale x 100)	Slow or unstable learning	Loss curve, saturation of tanh units
Fix 2	Standardize with training mean/std	Faster, smoother convergence	Compare with Problem 2

Each row changes only one factor from the previous stable state, so differences can be attributed to that factor. The root cause is classified as data (scaling), hyperparameter (learning rate) or training procedure,

with evidence from the table.

Cross-verification (Role B's explanation). Increasing the distance between relevant observations makes learning difficult for a conventional RNN because the gradient must cross many time steps, each multiplying it by a recurrent Jacobian. Gating or shorter paths shorten this route.

Cross-verification (Role A's explanation). The source of a failure must be identified before several components are changed together, because otherwise the effect of each correction cannot be separated. If two changes are made at once and the model improves, it is unknown which change was responsible, and a fix may hide the original fault.

Q7 - Design and Debug a Sequence Model from First Principles (10 marks)

Design scenario

A company predicts a future event from variable-length, noisy observation sequences with occasional missing values, limited labels, and both short- and long-term dependencies. The existing RNN shows unstable validation performance and inconsistent predictions.

Stage 1 - Initial design proposal (own draft)

Write your initial design proposal here before refining the design, and keep it unedited. It should cover architecture, input representation, sequence handling, output, initial hyperparameters, metrics, possible failure sources, preprocessing, training, validation, hyperparameter approach and debugging procedure. No marks are deducted for an incorrect initial design. Attach it as a separate page:

_____

Stage 2 - Design comparison record

The final column records the decision adopted in the final design, so the influence of the initial draft is visible.

Aspect	Initial draft	Alternative considered	Final design (adopted)
Architecture	_____	_____	Masked LSTM (64) -> Dropout -> LSTM (32) -> Dense (16) -> sigmoid
Input / preprocessing	_____	_____	Impute + missingness flag, normalize on training data only
Sequence handling	_____	_____	Pad to max length, mask padding, cap at 99th percentile
Metrics	_____	_____	PR-AUC, F1, recall, precision, confusion matrix
Hyperparameter approach	_____	_____	Baseline, then one-factor changes or random search
Debugging checks	_____	_____	Tiny-batch overfit, NaN guard, gradient norm, clipping

Comparison record. Where the designs agree: sequence model of the LSTM family, training-only normalization, early stopping and an untouched test set. Where they differ: the handling of missing values and the debugging order. Adopted: the missingness indicator and masking, with clipping and the NaN guard as the first debugging steps. Missing cases identified: class imbalance, leakage across sequences from the same machine, and the cost of false negatives.

Stage 3 - Final model design

Element	Decision
Problem definition	Predict whether a future event will occur from a variable-length observation sequence
Input representation	X in R^(T x d) with T variable; d sensor features plus d missingness flags, so 2d channels
Data preprocessing	Impute missing values with training mean; standardize with training mean and std; no test statistics
Sequence length strategy	Pad to the batch maximum (capped at the 99th percentile of training lengths, keeping the most recent steps); mask padding
Architecture	Masking -> LSTM(64, return sequences) -> Dropout(0.2) -> LSTM(32) -> Dense(16, ReLU) -> Dense(1, sigmoid)
Recurrent processing	Gated LSTM; recursive processing not used because the data is ordered in time, with no tree

Element	Decision
Hidden-state representation	64-dimensional h_t at each step; the final 32-dimensional h_T is the sequence summary
Output layer	One sigmoid unit giving P(event). Softmax with categorical cross-entropy if events are mutually exclusive
Activation functions	Sigmoid for gates and output; tanh for cell candidate and output; ReLU in the dense layer
Loss function	Binary cross-entropy, with class weights for imbalance
Performance metrics	PR-AUC (primary), F1 at a validation-chosen threshold, recall, precision, confusion matrix
Hyperparameters	Units 64/32; 2 layers; dropout 0.2; batch 32; Adam lr 0.001; clipnorm 1.0; early stopping patience 6
Training procedure	Mini-batches, per-epoch loss logging, checkpoint of the best validation loss, TerminateOnNaN
Validation procedure	Stratified train/validation/test split by sequence (no sequence in two sets); model selection on validation only
Debugging strategy	Shape and NaN checks; tiny-batch overfit test; gradient norm monitoring; one-factor experiments
Model-refinement procedure	Compare baseline RNN and refined model on the same split; keep a change only if the validation gain is repeatable across seeds

Architecture diagram.

```
Input (B, T, 2d)  ->  Masking  ->  LSTM(64, return_sequences)  ->  Dropout(0.2)
                  ->  LSTM(32)  ->  Dense(16, ReLU)  ->  Dense(1, sigmoid)  ->  P(event)
Padded steps are skipped by the mask; missing values are imputed; features are standardized.
```

Gated recurrent processing (per time step).

```
f_t = sigmoid(W_f [h_(t-1), x_t] + b_f) forget gate i_t = sigmoid(W_i [h_(t-1), x_t] + b_i) input gate o_t = sigmoid(W_o [h_(t-1), x_t] + b_o) output gate c~_t = tanh(W_c [h_(t-1), x_t] + b_c) candidate c_t = f_t * c_(t-1) + i_t * c~_t cell state h_t = o_t * tanh(c_t) hidden state
```

The additive cell update lets information persist when f_t is close to 1, which is why an LSTM is chosen for dependencies at different distances.

Stage	Output shape (B, T, d)	Purpose
Input (imputed, normalized, padded)	(B, T, 2d)	Padded steps are zero
Masking	(B, T, 2d)	Ignores padded steps in later layers
LSTM 1 (64, return sequences)	(B, T, 64)	Short- and long-term temporal patterns
Dropout 0.2	(B, T, 64)	Regularization
LSTM 2 (32)	(B, 32)	Final sequence summary
Dense 16, ReLU	(B, 16)	Non-linear decision features
Dense 1, sigmoid	(B, 1)	Probability of the future event

Pseudocode

- Load and validate sequences and labels (lengths, NaN counts, label values).
- Split into train, validation and test sets, stratified, with no sequence in two sets.
- Compute mean and std on training values only; impute and standardize all sets with them.
- Pad to a fixed maximum length, add missingness flags, and create the mask through the Masking layer.
- Build Masking -> LSTM -> Dropout -> LSTM -> Dense -> sigmoid.
- Train with Adam (clipnorm 1.0), weighted binary cross-entropy, mini-batches of 32.

- Record training and validation loss each epoch; stop early and restore the best checkpoint.
- Evaluate once on the test set: confusion matrix, precision, recall, F1, PR-AUC.
- Debug if needed: check gradient norms and NaN/Inf; change one factor at a time.
- Compare with the baseline under the same split and keep the refinement only if the gain is repeatable.

Implementation

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, average_precision_score
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.sequence import pad_sequences

# 1. Synthetic variable-length sequences with 5% missing values and ~20% events
rng = np.random.default_rng(42)
d, N = 4, 3000
seqs, labels = [], []
for _ in range(N):
    T = rng.integers(20, 80)
    x = rng.normal(0, 1, (T, d))
    event = rng.random() < 0.2
    if event:
        x[:T // 4, 0] += 1.5                      # early signal: long-term dependency
    x[rng.random((T, d)) < 0.05] = np.nan
    seqs.append(x); labels.append(float(event))
y = np.array(labels, dtype='float32')

# 2. Leak-free stratified split by sequence
idx = np.arange(N)
tr, te = train_test_split(idx, test_size=0.2, stratify=y, random_state=1)
tr, va = train_test_split(tr, test_size=0.25, stratify=y[tr], random_state=1)

# 3. Impute and standardize with TRAINING statistics only; add missingness flags
train_all = np.concatenate([seqs[i] for i in tr])
mean, std = np.nanmean(train_all, axis=0), np.nanstd(train_all, axis=0)
def prep(ids, max_len=80):
    out = []
    for i in ids:
        s = seqs[i]
        miss = np.isnan(s).astype('float32')
        s = np.where(np.isnan(s), mean, s)
        out.append(np.concatenate([(s - mean) / std, miss], axis=1))
    return pad_sequences(out, maxlen=max_len, dtype='float32', padding='post', value=0.0)
X_tr, X_va, X_te = prep(tr), prep(va), prep(te)
y_tr, y_va, y_te = y[tr], y[va], y[te]

# 4. Masked two-layer LSTM with sigmoid output
model = keras.Sequential([
    layers.Input(shape=X_tr.shape[1:]),
    layers.Masking(mask_value=0.0),
    layers.LSTM(64, return_sequences=True),
    layers.Dropout(0.2),
    layers.LSTM(32),
    layers.Dense(16, activation='relu'),
    layers.Dense(1, activation='sigmoid'),
])
model.compile(optimizer=keras.optimizers.Adam(1e-3, clipnorm=1.0),
              loss='binary_crossentropy',
              metrics=['accuracy', keras.metrics.AUC(curve='PR', name='pr_auc')])

# 5. Class weight, early stopping, checkpoint, NaN guard
w_event = (len(y_tr) - y_tr.sum()) / y_tr.sum()
callbacks = [keras.callbacks.EarlyStopping(monitor='val_loss', patience=6, restore_best_weights=True),
             keras.callbacks.ModelCheckpoint('best_lstm.keras', save_best_only=True),
             keras.callbacks.TerminateOnNaN()]
hist = model.fit(X_tr, y_tr, validation_data=(X_va, y_va), epochs=50, batch_size=32,
                 class_weight={0: 1.0, 1: float(w_event)}, callbacks=callbacks, verbose=2)

# 6. One final evaluation on the untouched test set
p_te = model.predict(X_te).ravel()
pred = (p_te >= 0.5).astype(int)
print(confusion_matrix(y_te, pred))
print(classification_report(y_te, pred, target_names=['no event', 'event']))
print('PR-AUC:', round(average_precision_score(y_te, p_te), 4))
```

The printed numbers depend on the synthetic data and seed. The threshold of 0.5 should be replaced by a value chosen on validation data for the real application.

Model comparison and evaluation records

Item	Baseline (Simple RNN)	Refined (masked LSTM)
Validation loss (best)	_____	_____
Validation PR-AUC / F1	_____	_____
Test F1 (once)	_____	_____
Training time per epoch (s)	_____	_____
Inference time (ms per sequence)	_____	_____
Peak memory	_____	_____

Stage 4 - Validation (both roles)

Role A - Mathematical and performance validation.

- **Computational flow.** Input (B, T, 2d) is masked, passed through the first LSTM to give a state at each step, through dropout, then through the second LSTM to a final 32-dimensional summary. The dense layers map this to one probability.
- **Components.** The mask makes padding invisible. Gates control what is kept in c_t; the output gate controls what is exposed in h_t. Dropout regularizes the first recurrent layer. The dense head turns the summary into a decision.
- **Loss.** Binary cross-entropy is the negative log-likelihood of a Bernoulli model; it gives calibrated probabilities. Class weights rescale the event term so rare events contribute appropriately.
- **Metric justification.** PR-AUC summarizes ranking under imbalance without a threshold. F1 and recall at a validation-chosen threshold match the cost of errors. Accuracy is reported but not used for selection.
- **Curves to inspect.** Training loss falling while validation loss rises indicates overfitting; both high indicates underfitting; spikes or NaN indicate unstable learning. Record which pattern appears in the run.
- **Against the baseline.** Compare validation and test metrics with the Simple RNN under the same split. The refined model is accepted only if the gain is consistent across seeds.

Role B - Experimental and debugging validation.

- **Correctness tests.** (1) Masking test: adding trailing zero steps must not change predictions. (2) Tiny-batch test: the model must overfit 32 sequences; failure suggests a bug. (3) NaN test: no NaN or Inf in inputs, loss or gradients. (4) Leakage test: normalization statistics must equal the training-set statistics.
- **Instrumentation.** Log training and validation loss, gradient norm per epoch, and per-threshold metrics.
- **Hyperparameter configurations.** Compare at least three: the baseline, a smaller model and a higher-dropout model, under the same seeds.
- **Before and after debugging.** Record metrics before the fix (for example with an unclipped learning rate of 0.01) and after the fix, in the same table format.
- **Greatest improvement.** Attribute the gain to one change using the one-factor results. Expected candidates are input normalization and gradient clipping; the evidence decides.

```
# Correctness test 1: padding must not change the prediction (Masking works)
p1 = model.predict(X_te[:5], verbose=0)
X_longer = np.concatenate([X_te[:5], np.zeros((5, 10, X_te.shape[2]), dtype='float32')], axis=1)
p2 = model.predict(X_longer, verbose=0)
assert np.allclose(p1, p2, atol=1e-5), 'padding changed the prediction'

# Correctness test 2: normalization uses training statistics only
assert np.allclose(mean, np.nanmean(np.concatenate([seqs[i] for i in tr]), axis=0))
```

Stage 5 - Cross-defence

Role A must explain: the experimental evidence that the final model is better than the original. Answer: the same split, seeds and metrics are used for both; the refined model must improve validation F1 or PR-AUC and loss by a margin larger than the seed-to-seed spread, and the test result must confirm the validation gain. An improvement on the training curve alone is not evidence. The original model is kept for comparison, and the test set is used once.

Role B must explain: how the architecture and hyperparameters shape training and validation behaviour. Answer: the gated LSTM and the dropout layer determine how much of the past is kept and how much the model can memorize, which controls the gap between training and validation loss. The learning rate and clipping determine whether the loss curve is smooth or oscillating. Batch size changes the gradient noise and therefore the validation curve's stability.

Stage 6 - Evaluation

- **Suitability for sequences.** The LSTM processes steps in order and keeps a gated memory, so it handles dependencies at different distances. The mask handles variable length without treating padding as data.
- **Metrics.** PR-AUC and F1 are appropriate because events are rare and the costs of false negatives and false alarms differ.
- **Most influential hyperparameters.** Learning rate and model capacity usually dominate, with dropout controlling the gap. The evidence from the one-factor experiments identifies which applies here.
- **Evidence of overfitting or underfitting.** Train-validation loss gap and the learning curves; the confusion matrix on validation data.
- **Most effective debugging strategy.** Stated as the single change with the greatest measured improvement in the before/after table.
- **Additional computational cost.** LSTM gates give about four times the parameters of a Simple RNN of the same width, and inference time grows with T. Masking and padding add memory.
- **When a recurrent model may not be appropriate.** When the data is a tree or has no temporal order, or when very long sequences make the cost unacceptable and a shorter representation or attention model is preferable.
- **When a recursive model may be preferable.** When a reliable hierarchy, such as a parse tree, is available and the composition of parts determines the label.
- **Generalization.** Refinements that reduce the train-validation gap (dropout, early stopping, a smaller model, missingness handling) improve generalization only when the validation gain holds on the test set.

Conclusion. A deep learning practitioner should examine: the data structure (order, hierarchy, length distribution); the label definition and class balance; the cost of each error type; the split design, including leakage across sequences from the same source; the sensitivity of the model to learning rate and capacity; the stability of training (NaN checks, gradient norms); the validation evidence across seeds; and the computational budget for training and inference. Only after these are checked should the architecture be changed.

Individual Work Record

Question	Work completed by the student
Q1	Sequence representation, RNN state equations and unfolded structure, worked state update, sequence-length experiment design, recurrent vs recursive comparison
Q2	Confusion-matrix metrics, training vs validation analysis, sequence-level metric choice, accuracy versus minority-class analysis

Question	Work completed by the student
Q3	Hyperparameter list and candidate values, controlled and random search design, interpretation of the best configuration
Q4	Symptom diagnosis, gradient clipping analysis, NaN debugging procedure, controlled before/after experiment design
Q5	RNN and recursive representations, structural assumptions, equivalent evaluation protocol, comparison table
Q6	Long-term dependency experiment design, vanishing-gradient analysis, controlled debugging experiment
Q7	Final masked-LSTM design, pseudocode, implementation, correctness tests, evaluation plan and defence

Student: A HARITHA **Register number:** 192321147